Manjot Singh
23BCS12549
KRG-2A

Exp-7

Problem statement

To find the cheapest flights from source to destination with at most K stops in between source and destination

Algorithm

1) create queue
2) Push (src, 0, 0)
   Also, keep minCost = INT_MAX
3) while queue not empty :
4) pop front
5) if node = dst && new Cost < min Cost :
   min Cost = new Cost .
6) if stops > K :
   continue
7) else :
   push adjacent nodes

Pseudocode

```
cheapestFlight (adj, n, src, dst, k) {
    queue q
    q.push( src, 0, 0)
    min Cost = INT_MAX.

    while !q.empty() {
(u, cost, stop) = q.pop ( q.front() )

    if u == dst
        min cost = min ( minCost, cost)
```

```
if stops > k :
    continue

for v = 0 to n - 1
    if adj [u][v] != 0
        newCost = cost + adj [u] [v]
    if newCost < minCost
        push (v, newCost, stops + 1)

if minCost == INT_MAX :
    return -1
return minCost
```

Testcases :

Input : n = 4 ,
flights = [[0,1,100], [1,2,100], [2,0,100], [1,3,600], [2,3,200]] ,
src = 0 , dst = 3 , k = 1

Output : 700

~~Comb~~

Complexities

Since we have adjacency list

Time complexity : $O(V^2)$
Space complexity : $O(V)$